📋 Cahier de Charge – Projet Heart Disease UCI

---

1. Contexte et Présentation du Projet

1.1 Contexte général

Les maladies cardiovasculaires représentent la **première cause de mortalité dans le monde**, avec environ 17,9 millions de décès par an selon l'OMS. Un diagnostic précoce et précis est essentiel pour améliorer la prise en charge des patients. L'intelligence artificielle et le Machine Learning offrent des outils puissants pour assister les médecins dans ce processus de diagnostic.

1.2 Objectif du projet

Développer un **modèle de classification binaire** capable de prédire la présence ou l'absence de maladie cardiaque chez un patient à partir de ses données cliniques. Le projet vise à :

- Explorer et comprendre les données médicales disponibles
- Identifier les facteurs les plus prédictifs de la maladie cardiaque
- Comparer plusieurs algorithmes de classification
- Optimiser les performances via le fine-tuning
- Sélectionner le meilleur modèle pour un déploiement potentiel

---

2. Description du Jeu de Données

2.1 Source

UCI Machine Learning Repository – Heart Disease Dataset
Données collectées dans **4 centres hospitaliers** :

Centre	Localisation	Pays
Cleveland Clinic	Cleveland, Ohio	USA
Hungarian Institute of Cardiology	Budapest	Hongrie
University Hospital	Zurich	Suisse
VA Medical Center	Long Beach, Californie	USA

2.2 Dimensions

- **920 observations** (patients)
- **16 variables** (dont 1 identifiant, 1 variable cible)
- **14 variables prédictives** utilisables

2.3 Dictionnaire des variables

#	Variable	Description	Type	Valeurs
1	`id`	Identifiant unique du patient	Identifiant	Entier (à supprimer)
2	`age`	Âge du patient	Numérique continue	28 – 77 ans
3	`sex`	Sexe	Catégorielle binaire	Male, Female
4	`dataset`	Centre hospitalier d'origine	Catégorielle nominale	Cleveland, Hungary, Switzerland, VA Long Beach
5	`cp`	Type de douleur thoracique	Catégorielle ordinale	typical angina, atypical angina, non-anginal, asymptomatic
6	`trestbps`	Pression artérielle au repos (mm Hg)	Numérique continue	0 – 200
7	`chol`	Cholestérol sérique (mg/dl)	Numérique continue	0 – 603
8	`fbs`	Glycémie à jeun > 120 mg/dl	Booléenne	TRUE, FALSE
9	`restecg`	Résultats ECG au repos	Catégorielle nominale	normal, lv hypertrophy, st-t abnormality
10	`thalch`	Fréquence cardiaque maximale atteinte	Numérique continue	60 – 202
11	`exang`	Angine induite par l'exercice	Booléenne	TRUE, FALSE
12	`oldpeak`	Dépression ST induite par l'exercice	Numérique continue	-2.6 – 6.2
13	`slope`	Pente du segment ST à l'effort	Catégorielle ordinale	upsloping, flat, downsloping
14	`ca`	Nb de vaisseaux majeurs colorés (fluoroscopie)	Numérique discrète	0, 1, 2, 3
15	`thal`	Thalassémie	Catégorielle nominale	normal, fixed defect, reversable defect
16	`num`	**Variable cible** – Diagnostic	Catégorielle ordinale	0 = sain, 1–4 = malade

2.4 Variable cible

La variable `num` prend les valeurs 0 à 4 (degré de sévérité). Pour notre projet, elle sera **binarisée** :

Valeur originale	Classe binaire	Signification
0	**0**	Patient **sain**
1, 2, 3, 4	**1**	Patient **malade**

2.5 Problèmes identifiés dans les données

Problème	Variables concernées	Impact
Valeurs manquantes	`ca` (~65%), `slope` (~33%), `thal` (~30%), `chol`, `trestbps`, `thalch`, `oldpeak`, `fbs`, `restecg`	Imputation nécessaire
Valeurs aberrantes	`chol = 0`, `trestbps = 0`	Physiologiquement impossible
Données hétérogènes	`dataset` (4 sources différentes)	Variabilité inter-centres
Variables catégorielles textuelles	`sex`, `cp`, `restecg`, `slope`, `thal`, `fbs`, `exang`	Encodage nécessaire

---

3. Étapes du Projet (Pipeline Détaillé)

📌 Étape 1 : Importation et Chargement

Objectif : Charger les bibliothèques et le dataset.

- Importer les bibliothèques : `pandas`, `numpy`, `matplotlib`, `seaborn`, `sklearn`, `xgboost`, `statsmodels`
- Charger le fichier CSV `heart_disease_uci.csv`
- Vérifier les dimensions (`shape`), les types (`dtypes`), et afficher un aperçu (`head`, `info`)

📌 Étape 2 : Analyse Exploratoire des Données (EDA)

Objectif : Comprendre la structure, la distribution et les relations entre les variables.

2.1 Statistiques descriptives

- `describe()` pour les variables numériques (moyenne, écart-type, quartiles)
- `describe(include='object')` pour les variables catégorielles (fréquences)

2.2 Analyse des valeurs manquantes

- Calculer le nombre et le pourcentage de valeurs manquantes par colonne
- Visualiser avec un barplot horizontal

2.3 Distribution de la variable cible

- Countplot de `num` (distribution originale 0-4)
- Pie chart de la variable binarisée `target` (sain vs malade)
- Vérifier l'équilibre des classes

2.4 Distributions des variables numériques

- Histogrammes avec KDE pour : `age`, `trestbps`, `chol`, `thalch`, `oldpeak`, `ca`
- Ajouter les lignes de moyenne et médiane
- Identifier les distributions asymétriques

2.5 Détection des outliers

- Boxplots pour chaque variable numérique
- Identifier les valeurs extrêmes (IQR)
- Repérer les valeurs aberrantes (`chol=0`, `trestbps=0`)

2.6 Distribution des variables catégorielles

- Countplots pour : `sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `thal`, `dataset`
- Annoter les effectifs sur chaque barre

2.7 Matrice de corrélation

- Encoder temporairement les variables catégorielles
- Calculer et afficher la matrice de corrélation (heatmap triangulaire)
- Identifier les variables fortement corrélées avec la cible

2.8 Relations variables vs cible

- Boxplots : variables numériques groupées par `target`
- Stacked barplots : variables catégorielles vs `target` (proportions)

📌 Étape 3 : Nettoyage et Prétraitement

Objectif : Préparer les données pour la modélisation.

3.1 Suppression des colonnes non pertinentes

- Supprimer `id` (identifiant, aucune valeur prédictive)
- Supprimer `dataset` (origine, peut introduire un biais)
- Supprimer `num` (remplacé par `target` binaire)

3.2 Traitement des valeurs manquantes

- **Variables numériques** (`trestbps`, `chol`, `thalch`, `oldpeak`, `ca`) : imputation par la **médiane** (robuste aux outliers)
- **Variables catégorielles** (`slope`, `thal`, `fbs`, `restecg`, `exang`) : imputation par le **mode** (valeur la plus fréquente)

3.3 Traitement des valeurs aberrantes

- `chol = 0` → remplacer par la médiane des valeurs > 0
- `trestbps = 0` → remplacer par la médiane des valeurs > 0

3.4 Encodage des variables catégorielles

- Appliquer `LabelEncoder` sur : `sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `thal`
- Documenter le mapping d'encodage pour chaque variable

3.5 Séparation Features / Target

- `X` = toutes les colonnes sauf `target`
- `y` = colonne `target`

3.6 Split Train / Test

- Ratio : **80% train / 20% test**
- `stratify=y` pour conserver les proportions de classes
- `random_state=42` pour la reproductibilité

3.7 Standardisation

- Appliquer `StandardScaler` (moyenne=0, écart-type=1)
- `fit_transform` sur le train, `transform` sur le test
- Conserver les noms de colonnes via `pd.DataFrame`

📌 Étape 4 : Modélisation – Modèles Baseline

Objectif : Entraîner 8 modèles avec les hyperparamètres par défaut.

#	Modèle	Algorithme	Particularité
1	Logistic Regression (L2)	Régression logistique	Régularisation Ridge
2	Logistic Regression (L1)	Régression logistique	Régularisation Lasso (sélection de variables)
3	Logistic Regression (ElasticNet)	Régression logistique	Combinaison L1 + L2
4	SVC	Support Vector Machine	Séparation par hyperplan optimal

#	Modèle	Algorithme	Particularité
5	KNN	K plus proches voisins	Classification par voisinage
6	Decision Tree	Arbre de décision	Règles de décision interprétables
7	Random Forest	Ensemble d'arbres	Bagging + vote majoritaire
8	XGBoost	Gradient Boosting	Boosting séquentiel

Pour chaque modèle :

- Entraîner sur `X_train_scaled`
- Prédire sur `X_test_scaled`
- Calculer les probabilités (`predict_proba`)

📌 Étape 5 : Évaluation des Modèles

Objectif : Évaluer et comparer les performances avec des métriques adaptées à la classification.

5.1 Matrice de confusion

- Pour chaque modèle, afficher la matrice 2×2 : `[[VN, FP], [FN, VP]]`
- Heatmap annotée (8 matrices en grille)

5.2 Classification Report

- Pour chaque modèle : **Precision**, **Recall**, **F1-Score** par classe
- Macro/Weighted average

5.3 Courbes ROC et AUC

- Tracer la courbe ROC (TPR vs FPR) pour chaque modèle
- Calculer le **ROC AUC Score**
- Superposer toutes les courbes sur un même graphique

5.4 Tableau récapitulatif

- Tableau comparatif : Accuracy, Precision, Recall, F1-Score, ROC AUC
- Trier par ROC AUC décroissant

📌 Étape 6 : Sélection des Caractéristiques – Backward Elimination

Objectif : Identifier les variables statistiquement significatives.

6.1 Explication théorique (cellule markdown)

- Principe de la méthode Backward Elimination (pas à pas)

- Définition et interprétation de la **p-value**
- Seuil de significativité ($\alpha = 0.05$)
- Hypothèses H_0 et H_1

6.2 Implémentation

- Utiliser `statsmodels.OLS` avec constante
- Boucle itérative : supprimer la variable avec la plus grande p-value > 0.05
- Afficher les p-values à chaque étape
- Tracer l'historique (tableau des étapes)

6.3 Re-test des modèles

- Ré-entraîner les 8 modèles avec les variables sélectionnées uniquement
- Comparer les résultats (tableau + graphiques) : toutes variables vs sélectionnées
- Analyser l'impact de la réduction dimensionnelle

📌 Étape 7 : Fine-Tuning (GridSearchCV)

Objectif : Optimiser les hyperparamètres de chaque modèle.

7.1 Définition des grilles

Pour chaque modèle, définir une grille d'hyperparamètres à explorer :

Modèle	Hyperparamètres explorés
LogReg (L2)	`C` : [0.01, 0.1, 1, 10, 100]
LogReg (L1)	`C` : [0.01, 0.1, 1, 10, 100]
LogReg (ElasticNet)	`C`, `l1_ratio`
SVC	`C`, `kernel`, `gamma`
KNN	`n_neighbors`, `weights`, `metric`
Decision Tree	`max_depth`, `min_samples_split`, `min_samples_leaf`
Random Forest	`n_estimators`, `max_depth`, `min_samples_split`
XGBoost	`n_estimators`, `max_depth`, `learning_rate`, `subsample`

7.2 Exécution

- `GridSearchCV` avec **5-folds cross-validation**
- Scoring : **ROC AUC**
- `n_jobs=-1` (parallélisation)

7.3 Évaluation des modèles tunés

- Matrices de confusion (heatmaps)
- Classification reports
- Courbes ROC superposées
- Afficher les meilleurs hyperparamètres pour chaque modèle

📌 Étape 8 : Comparaison Finale – Baseline vs Tuned

Objectif : Quantifier l'amélioration apportée par le fine-tuning.

- Tableau comparatif : Accuracy, F1-Score, ROC AUC (Baseline vs Tuned + Δ)
- Graphiques barplot côte à côte (3 métriques)
- Courbes ROC : Baseline vs Tuned (2 graphiques côte à côte)
- Identifier le **meilleur modèle global**

📌 Étape 9 : Conclusion et Recommandations

- Synthèse des résultats
- Meilleur modèle identifié et justification
- Limites du projet (taille du dataset, données multi-centres, binarisation)
- Perspectives : déploiement, collecte de données supplémentaires, deep learning

---

4. Livrables

#	Livrable	Format
1	Cahier de charge	`cahier_de_charge.md`
2	Notebook complet	`heart_disease_analysis.ipynb`
3	Dataset source	`heart_disease_uci.csv`

---

5. Outils et Technologies

Catégorie	Outils
Langage	Python 3.x
Data	pandas, numpy
Visualisation	matplotlib, seaborn
ML	scikit-learn, xgboost
Statistiques	statsmodels

Catégorie	Outils
Environnement	Jupyter Notebook